Backend Schema Document: Cancer Care & Cure Foundation Website

Version 1.0 (Draft) | 24 October 2026 | Companion to the PRD, TRD and App Flow | Status: For review

1. Purpose and Scope

This document defines the database schema, access rules and data lifecycle for the website backend. It implements the data model in Section 8 of the TRD and the form and donation flows in the App Flow document.

Database: PostgreSQL on Supabase. All tables live in the `public` schema unless stated.

Phase 1 tables: contact submissions, volunteer applications, staff profiles, staff notes, email events, rate limit hits, data requests, audit log.

Phase 2 tables: donations, payment webhook events, receipt counters.

Out of scope: Patient records, medical documents, identity numbers and bank details. The forms must never collect them, and the schema deliberately has no columns for them.

2. Design Principles

- **Store first, notify second.** A submission is written to the database before any email is sent, so no enquiry is lost.
- **Minimum data.** Only fields needed to respond are stored. No medical details, ID numbers or bank details.
- **Private by default.** Row-level security is on for every table and there are no public policies. Only the server (service role) writes; only authorised staff read.
- **Consent is recorded.** Every submission stores when consent was given and which privacy policy version applied.
- **Auditable and deletable.** Staff actions are logged, and records can be deleted or anonymised on request or on expiry.
- **Portable.** Standard Postgres features only, so the database can move providers without a rewrite.

3. Entity Overview

Table	Purpose	Phase
`contact_submissions`	Enquiries from the Contact page, including patient assistance, CSR and donation enquiries	1
`volunteer_applications`	Applications from the Volunteer page	1
`staff_profiles`	Which signed-in users are staff, and their role	1
`staff_notes`	Internal notes attached to a submission or application	1
`email_events`	Log of notification emails, with retry state	1
`rate_limit_hits`	Request log used for simple rate limiting	1
`data_requests`	Access, correction and deletion requests from the public	1
`audit_log`	Record of staff and system actions on personal data	1
`donations`	One row per donation attempt	2
`payment_webhook_events`	Raw gateway webhook events for idempotency and debugging	2
`receipt_counters`	Sequential receipt numbering per financial year	2

Relationships: `staff_notes` points to a contact submission or a volunteer application. `email_events` points to the submission it relates to. `donations` has many `payment_webhook_events`. `staff_profiles` links to Supabase `auth.users`. `audit_log` references staff by user id.

4. Enumerated Types

```
create type enquiry_type as enum (
  'patient_assistance', 'csr_partnership', 'donation',
  'volunteering', 'media', 'other'
);

create type record_status as enum ('new', 'in_progress', 'closed');

create type volunteer_role as enum (
  'event_support', 'awareness', 'patient_support',
```

```
  'administration', 'digital_and_design', 'other'
);

create type availability as enum ('weekdays', 'weekends', 'flexible');

create type staff_role as enum ('admin', 'staff');

create type email_status as enum ('pending', 'sent', 'failed');

create type request_type as enum ('access', 'correction', 'deletion');

create type donation_status as enum (
  'created', 'pending', 'paid', 'failed', 'refunded'
);
```

Enumerated types match the dropdown values in the TRD form tables. Adding a value later is a simple migration.

5. Phase 1 Tables

5.1 contact_submissions

```
create table contact_submissions (
  id              uuid primary key default gen_random_uuid(),
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  full_name       text not null check (char_length(full_name) between 2 and
100),
  email           text not null check (char_length(email) <= 254),
  phone           text check (phone ~ '^[0-9+()\- ]{7,20}$'),
  enquiry_type    enquiry_type not null,
  message         text not null check (char_length(message) between 10 and
2000),
  consent_at      timestamptz not null,
  consent_version text not null,
  status          record_status not null default 'new',
  assigned_to     uuid references staff_profiles(user_id),
  ip_hash         text,
  closed_at       timestamptz,
  delete_after    date
);

create index contact_status_created_idx on contact_submissions (status,
created_at desc);
```

```
create index contact_type_idx on contact_submissions (enquiry_type);
create index contact_email_idx on contact_submissions (lower(email));
create index contact_delete_after_idx on contact_submissions (delete_after)
  where delete_after is not null;
```

Column	Notes
`consent_at`, `consent_version`	When the person ticked the consent box and which policy version was live
`ip_hash`	Salted hash of the IP address for abuse control; the raw IP is never stored
`assigned_to`	Staff member handling the enquiry
`delete_after`	Date after which the record is deleted or anonymised under the retention policy

The phone check allows digits, plus, brackets, hyphen and spaces. The server also validates and normalises numbers before insert.

5.2 volunteer_applications

```
create table volunteer_applications (
  id              uuid primary key default gen_random_uuid(),
  created_at      timestamptz not null default now(),
  updated_at      timestamptz not null default now(),
  full_name       text not null check (char_length(full_name) between 2 and
100),
  email           text not null check (char_length(email) <= 254),
  phone           text not null check (phone ~ '^[0-9+()\- ]{7,20}$'),
  city            text not null check (char_length(city) between 2 and 80),
  role            volunteer_role not null,
  skills          text check (char_length(skills) <= 500),
  availability    availability not null,
  consent_at      timestamptz not null,
  consent_version text not null,
  status          record_status not null default 'new',
  assigned_to     uuid references staff_profiles(user_id),
  ip_hash         text,
  closed_at       timestamptz,
  delete_after    date
);

create index volunteer_status_created_idx on volunteer_applications (status,
created_at desc);
```

```
create index volunteer_role_city_idx on volunteer_applications (role, city);
create index volunteer_delete_after_idx on volunteer_applications
(delete_after)
  where delete_after is not null;
```

5.3 staff_profiles

```
create table staff_profiles (
  user_id    uuid primary key references auth.users(id) on delete cascade,
  full_name  text not null,
  role       staff_role not null default 'staff',
  active     boolean not null default true,
  created_at timestamptz not null default now()
);
```

Only users listed here (with `active = true`) can read or update submissions. Multi-factor authentication is required on all staff accounts in the Supabase project settings.

5.4 staff_notes

```
create table staff_notes (
  id          uuid primary key default gen_random_uuid(),
  created_at  timestamptz not null default now(),
  author_id   uuid not null references staff_profiles(user_id),
  record_type text not null check (record_type in ('contact', 'volunteer')),
  record_id   uuid not null,
  note        text not null check (char_length(note) between 1 and 2000)
);

create index staff_notes_record_idx on staff_notes (record_type, record_id,
created_at);
```

This replaces the single `notes` column suggested in the TRD, so each note keeps its author and time. Notes must not contain medical details; staff are told this in the dashboard guidance. A scheduled job removes orphaned notes when a record is deleted.

5.5 email_events

```
create table email_events (
  id            uuid primary key default gen_random_uuid(),
  created_at    timestamptz not null default now(),
  record_type   text not null check (record_type in ('contact', 'volunteer',
'donation')),
  record_id     uuid not null,
  purpose       text not null check (purpose in ('staff_notification',
'acknowledgement', 'receipt')),
  status        email_status not null default 'pending',
  attempts      int not null default 0,
  last_error    text,
  next_attempt  timestamptz,
  sent_at       timestamptz
);

create index email_events_due_idx on email_events (next_attempt) where status
= 'pending';
```

A background job (scheduled function) retries pending rows with backoff. `last_error` must never contain message bodies or personal data.

5.6 rate_limit_hits

```
create table rate_limit_hits (
  id         bigserial primary key,
  created_at timestamptz not null default now(),
  route      text not null,
  ip_hash    text not null
);

create index rate_limit_lookup_idx on rate_limit_hits (route, ip_hash,
created_at desc);
```

Rate limiting rule from the TRD: for example, 5 requests per IP per 10 minutes per route. Rows older than a day are deleted by a scheduled job. If traffic grows, move this to a managed key-value store.

5.7 data_requests

```
create table data_requests (
  id           uuid primary key default gen_random_uuid(),
  created_at   timestamptz not null default now(),
  email        text not null,
  type         request_type not null,
  details      text check (char_length(details) <= 1000),
  status       record_status not null default 'new',
  due_by       date,
  resolved_at  timestamptz,
  handled_by   uuid references staff_profiles(user_id)
);
```

This tracks requests to access, correct or delete personal data under India's Digital Personal Data Protection Act, 2023. Identity must be verified by staff before any data is shared or deleted. The response deadline in `due_by` is set with legal advice.

5.8 audit_log

```
create table audit_log (
  id          bigserial primary key,
  created_at  timestamptz not null default now(),
  actor_id    uuid references staff_profiles(user_id),
  action      text not null,
  record_type text,
  record_id   uuid,
  detail      jsonb not null default '{}'::jsonb
);

create index audit_log_record_idx on audit_log (record_type, record_id,
created_at desc);
```

Logged actions include status changes, assignment, note creation, export, deletion and anonymisation. `detail` stores field names and ids, never the personal data itself.

6. Phase 2 Tables (Donations)

Create these only after legal registration, bank setup and receipt rules are confirmed in writing.

6.1 donations

```
create table donations (
  id                 uuid primary key default gen_random_uuid(),
  created_at         timestamptz not null default now(),
  updated_at         timestamptz not null default now(),
  amount_paise       bigint not null check (amount_paise > 0),
  currency           char(3) not null default 'INR',
  status             donation_status not null default 'created',
  donor_name         text not null,
  donor_email        text not null,
  donor_phone        text,
  donor_pan_enc      bytea,
  gateway            text not null check (gateway in ('razorpay',
'cashfree')),
  gateway_order_id   text not null unique,
  gateway_payment_id text unique,
  receipt_number     text unique,
  receipt_issued_at  timestamptz,
  consent_at         timestamptz not null,
  consent_version    text not null
);

create index donations_status_created_idx on donations (status, created_at
desc);
create index donations_email_idx on donations (lower(donor_email));
```

Column	Notes
`amount_paise`	Integer amount in paise to avoid rounding errors
`donor_pan_enc`	Encrypted PAN, stored only if required for receipts; use database-level encryption (for example, Supabase Vault or pgcrypto) with the key kept outside the table
`gateway_order_id`, `gateway_payment_id`	Unique so a repeated event cannot create a duplicate donation
`receipt_number`	Sequential number from `receipt_counters`; 80G wording on the receipt only if the foundation holds valid approval

Card and UPI details are never stored; the hosted checkout keeps them at the gateway.

6.2 payment_webhook_events

```
create table payment_webhook_events (
  id            uuid primary key default gen_random_uuid(),
  received_at   timestamptz not null default now(),
  gateway       text not null,
  event_id      text not null,
  event_type    text not null,
  signature_ok  boolean not null,
  donation_id   uuid references donations(id),
  payload       jsonb not null,
  processed_at  timestamptz,
  unique (gateway, event_id)
);
```

The unique pair makes webhook handling idempotent: if the gateway retries an event, the insert conflicts and the handler returns success without changing the donation twice. A donation is marked paid only when `signature_ok` is true.

6.3 receipt_counters

```
create table receipt_counters (
  financial_year text primary key,
  last_number    int not null default 0
);
```

Receipt numbers are taken inside a transaction (`update ... set last_number = last_number + 1 returning last_number`) so numbers are sequential and never reused.

7. Row-Level Security

All tables have RLS enabled. The server uses the service role key, which bypasses RLS, only from route handlers and scheduled jobs. The browser never receives that key.

```
alter table contact_submissions enable row level security;
alter table volunteer_applications enable row level security;
alter table staff_profiles enable row level security;
alter table staff_notes enable row level security;
alter table email_events enable row level security;
alter table rate_limit_hits enable row level security;
alter table data_requests enable row level security;
alter table audit_log enable row level security;
-- Phase 2: donations, payment_webhook_events, receipt_counters

create function is_staff() returns boolean
```

```
  language sql stable security definer set search_path = public as $$
  select exists (
    select 1 from staff_profiles
    where user_id = auth.uid() and active
  );
$$;

create function is_admin() returns boolean
  language sql stable security definer set search_path = public as $$
  select exists (
    select 1 from staff_profiles
    where user_id = auth.uid() and active and role = 'admin'
  );
$$;

create policy staff_read_contact on contact_submissions
  for select to authenticated using (is_staff());
create policy staff_update_contact on contact_submissions
  for update to authenticated using (is_staff()) with check (is_staff());

create policy staff_read_volunteer on volunteer_applications
  for select to authenticated using (is_staff());
create policy staff_update_volunteer on volunteer_applications
  for update to authenticated using (is_staff()) with check (is_staff());

create policy staff_read_notes on staff_notes
  for select to authenticated using (is_staff());
create policy staff_add_notes on staff_notes
  for insert to authenticated with check (is_staff() and author_id =
auth.uid());

create policy admin_manage_staff on staff_profiles
  for all to authenticated using (is_admin()) with check (is_admin());
create policy staff_read_self on staff_profiles
  for select to authenticated using (user_id = auth.uid());

create policy staff_read_requests on data_requests
  for select to authenticated using (is_staff());
create policy staff_update_requests on data_requests
  for update to authenticated using (is_staff()) with check (is_staff());

create policy admin_read_audit on audit_log
  for select to authenticated using (is_admin());
```

Table	Public (anon)	Staff	Admin	Server (service role)
contact_submissions	None	Read, update status	Same as staff	Insert
volunteer_applications	None	Read, update status	Same as staff	Insert
staff_notes	None	Read, add own notes	Same as staff	None needed
staff_profiles	None	Read own row	Manage	Read
email_events	None	None	None	Full
rate_limit_hits	None	None	None	Full
data_requests	None	Read, update	Same as staff	Insert
audit_log	None	None	Read	Insert
donations (Phase 2)	None	Read only	Read only	Full
payment_webhook_events (Phase 2)	None	None	None	Full

Staff cannot delete rows directly. Deletion and anonymisation run through a controlled function or admin-only job so each one is audited.

8. Triggers and Functions

```
create function set_updated_at() returns trigger
  language plpgsql as $$
begin
  new.updated_at = now();
  return new;
end;
$$;

create trigger contact_updated before update on contact_submissions
  for each row execute function set_updated_at();
create trigger volunteer_updated before update on volunteer_applications
  for each row execute function set_updated_at();
```

```
create function set_closed_at() returns trigger
  language plpgsql as $$
begin
  if new.status = 'closed' and old.status <> 'closed' then
    new.closed_at = now();
  end if;
  return new;
end;
$$;

create trigger contact_closed before update on contact_submissions
  for each row execute function set_closed_at();
create trigger volunteer_closed before update on volunteer_applications
  for each row execute function set_closed_at();
```

Audit trigger: An after-update trigger on submissions and applications writes a row to `audit_log` when `status` or `assigned_to` changes, recording the old and new values and `auth.uid()`.

Anonymise function: `anonymise_contact(id)` and `anonymise_volunteer(id)` replace name, email, phone and free text with placeholders, keep the enquiry type, status and dates for statistics, and write an audit row. Restricted to admins and scheduled jobs.

9. Data Retention and Deletion

The periods below are proposals to be agreed with the foundation and a legal adviser before launch.

Data	Proposed retention	Method
Closed contact submissions	Set `delete_after` when closed (for example, 12 months)	Scheduled job anonymises or deletes after the date
Open submissions	Reviewed regularly; closed or deleted if no response is possible	Staff action
Volunteer applications	Set `delete_after` when closed (for example, 12 months)	Scheduled job
Staff notes	Deleted with their record	Cascade in the delete function
rate_limit_hits	1 day	Scheduled job
email_events	90 days	Scheduled job

Data	Proposed retention	Method
audit_log	Longer period agreed with legal advice	Admin-only purge
Donations (Phase 2)	As required by accounting and tax rules	Not auto-deleted; restricted access

Scheduled jobs run using Supabase `pg_cron` or scheduled edge functions. Each run writes a summary to `audit_log` (counts only).

A deletion request follows this path: record in `data_requests`, verify identity, run the delete or anonymise function for matching rows in each table, log the action, confirm to the requester, close the request.

10. API-to-Database Mapping

Endpoint	Reads	Writes	Notes
`POST /api/contact`	`rate_limit_hits`	`contact_submissions`, `email_events`, `rate_limit_hits`	Insert the record, then queue the email event; return success even if the email fails
`POST /api/volunteer`	`rate_limit_hits`	`volunteer_applications`, `email_events`, `rate_limit_hits`	Same flow as contact
`POST /api/data-request` (optional)	`rate_limit_hits`	`data_requests`, `rate_limit_hits`	Public form for access, correction and deletion requests
`POST /api/donations` (Phase 2)	None	`donations`	Creates the gateway order and stores `gateway_order_id`
`POST /api/webhooks/payments` (Phase 2)	`donations`	`payment_webhook_events`, `donations`, `receipt_counters`, `email_events`	Verify signature first; idempotent on `(gateway, event_id)`

In the contact flow the sequence is: validate, check bot token, check rate limit, insert the record (this must succeed), insert the email event, then try to send the email and update the event row.

11. Migrations and Environments

- Keep every schema change in a numbered SQL migration file in the repository (for example, `supabase/migrations/`); never change production by hand.
- Order for the first migration: enums, `staff_profiles`, submissions and applications, supporting tables, indexes, functions and triggers, then RLS policies.
- Use separate Supabase projects for development, preview and production; never copy production personal data into development.
- Seed development with fake data only.
- Review RLS policies in every pull request that touches the schema, and test them with both an anonymous and a staff session.
- Generate TypeScript types from the schema and share them with the route handlers and the Zod validation layer so the form fields and columns stay in step.

12. Backups, Security and Monitoring

- Enable automated daily backups and point-in-time recovery if the plan allows; test a restore before launch and after major changes.
- Service role key stored only as a server environment variable; rotate it when staff or developers change.
- Staff accounts use multi-factor authentication; remove access promptly when someone leaves (`active = false`).
- Restrict database network access where possible and use SSL connections.
- No personal data in logs or error messages; scrub request bodies in error tracking.
- Monitor failed email events, repeated rate limit hits, and unusual spikes in submissions.
- Review the audit log monthly during the first quarter after launch.

13. Example Queries for Staff and Reporting

```
-- New enquiries waiting for a response
select id, created_at, enquiry_type, full_name
from contact_submissions
where status = 'new'
order by created_at;

-- Enquiries by type this month (no personal data)
select enquiry_type, count(*)
from contact_submissions
where created_at >= date_trunc('month', now())
group by enquiry_type;
```

```
-- Volunteer pipeline by role and city
select role, city, count(*)
from volunteer_applications
where status <> 'closed'
group by role, city
order by count(*) desc;

-- Emails that failed and need attention
select id, record_type, purpose, attempts, last_error
from email_events
where status = 'failed';
```